ENTREGA 3: COMUNICAÇÃO MQTT E CONTROLE REMOTO

Sensor de Alarme com Detector de Movimento (PIR)

Eduardo Octavio de Paula Souza

1. INTRODUÇÃO

Esta entrega documenta a implementação da comunicação MQTT bidirecional e do controle remoto do projeto Sensor de Alarme com Detector de Movimento (PIR). O objetivo foi configurar o cliente MQTT no ESP32 para publicação de telemetria e recebimento de comandos, definir a hierarquia de tópicos e o formato das mensagens em JSON, e demonstrar o acionamento remoto dos atuadores (LEDs e buzzer) a partir de um comando enviado ao broker.

2. MONTAGEM FÍSICA (HARDWARE)

2.1 Componentes utilizados

• ESP32 DevKit (30 pinos, USB-C, módulo WROOM-32)

• Sensor PIR HC-SR501

• LED verde e LED vermelho, com resistores de 220Ω

• Buzzer ativo 3-24V (modelo 2312YX)

• Protoboard de 400 pinos e jumpers

2.2 Conexões realizadas

• PIR: VCC no pino VIN (5V) do ESP32, GND no trilho negativo, OUT no GPIO 13 (D13).

• LED verde: ânodo (+) ligado ao GPIO 12 (D12) através de resistor de 220Ω; cátodo (-) no trilho negativo.

• LED vermelho: ânodo (+) ligado ao GPIO 14 (D14) através de resistor de 220Ω; cátodo (-) no trilho negativo.

• Buzzer: terminal positivo ligado diretamente ao GPIO 27 (D27); terminal negativo no trilho negativo.

O sensor PIR foi realimentado com 5V (pino VIN), tensão nominal recomendada pelo fabricante do HC-SR501, garantindo maior alcance e estabilidade de detecção. O buzzer, por ser um componente de maior consumo de corrente, foi validado individualmente junto a uma fonte externa (pilha) antes da integração, confirmando o funcionamento do componente físico previamente à montagem final.

3. CONFIGURAÇÃO DO CLIENTE MQTT

O ESP32 foi configurado como cliente MQTT utilizando a biblioteca PubSubClient, conectando-se ao broker Mosquitto (executado via Docker) no endereço 192.168.2.168, porta 1883. Foram implementadas duas funcionalidades principais de comunicação:

• Publicação (publish): envio de telemetria do sensor PIR no tópico alarme/sensor/pir.

• Assinatura (subscribe): recebimento de comandos de controle no tópico alarme/comando/sistema, processados por uma função de callback.

A reconexão ao broker é verificada continuamente no loop principal, seguindo a mesma lógica de resiliência já validada para a conexão Wi-Fi, garantindo que o dispositivo se reconecte automaticamente em caso de queda de comunicação.

4. PLANO DE TÓPICOS E ESTRUTURA DAS MENSAGENS

4.1 Tópico de telemetria: alarme/sensor/pir

Direção: ESP32 → Broker. Publicado sempre que há mudança de estado do sensor PIR.

```
{
  "deviceId": "esp32-alarme-01",
  "sensorPIR": true,
  "timestamp": 74428
}
```

4.2 Tópico de comando: alarme/comando/sistema

Direção: Broker → ESP32. O ESP32 assina este tópico e reage a duas mensagens de texto simples:

```
"ON"   -> ativa o sistema (monitoramento e atuadores)
"OFF"  -> desativa o sistema (modo de espera)
```

Optou-se por um payload de texto simples (ao invés de um objeto JSON) para este tópico, dado que o comando representa uma única informação booleana de estado (ligado/desligado). A estrutura em JSON foi reservada para as mensagens de telemetria, que carregam múltiplos campos (identificação do dispositivo, valor do sensor e timestamp).

5. LÓGICA DE FUNCIONAMENTO DO SISTEMA

O comportamento do dispositivo foi estruturado em estados, controlados remotamente via MQTT:

1. Inicialização: ao ligar, o ESP32 conecta-se ao Wi-Fi e ao broker MQTT, e realiza a calibração do sensor PIR (30 segundos), necessária para o correto funcionamento do HC-SR501.

2. Pronto para uso: concluída a calibração, os LEDs verde e vermelho piscam simultaneamente por 1 segundo, sinalizando visualmente que o sistema está pronto e aguardando o comando de ativação. Em seguida, o dispositivo permanece em modo de espera (todos os atuadores desligados) até receber o comando ON.

3. Sistema ativado (ON): o LED verde acende e o monitoramento do sensor PIR é iniciado.

4. Movimento detectado: o LED vermelho acende (o verde apaga) e o buzzer é acionado, permanecendo neste estado enquanto o PIR detectar movimento.

5. Ausência de movimento: o sistema retorna ao LED verde aceso e o buzzer é desligado.

6. Sistema desativado (OFF): a qualquer momento, o recebimento do comando OFF desliga todos os atuadores (LEDs e buzzer) e retorna o dispositivo ao modo de espera, independentemente do estado do sensor PIR.

6. CÓDIGO-FONTE (ESP32)

Código final integrando conexão Wi-Fi com reconexão automática, cliente MQTT (publicação e assinatura), leitura do sensor PIR e controle dos atuadores conforme a lógica de estados descrita na seção anterior.

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "NOME_DA_REDE";
const char* password = "SENHA_DA_REDE";

const char* mqtt_server = "192.168.2.168";
const int mqtt_port = 1883;
const char* mqtt_client_id = "esp32-alarme-01";
const char* topico_telemetria = "alarme/sensor/pir";
const char* topico_comando_sistema = "alarme/comando/sistema";

const int pinoPIR = 13;
const int pinoLedVerde = 12;
const int pinoLedVermelho = 14;
const int pinoBuzzer = 27;

WiFiClient espClient;
PubSubClient client(espClient);

unsigned long ultimaTentativaReconexaoWifi = 0;
const long intervaloReconexaoWifi = 5000;
unsigned long ultimaTentativaReconexaoMqtt = 0;
const long intervaloReconexaoMqtt = 5000;

int estadoAnteriorPIR = LOW;
bool sistemaAtivo = false;

void setup() {
  Serial.begin(115200);
```

```
  delay(1000);
  pinMode(pinoPIR, INPUT);
  pinMode(pinoLedVermelho, OUTPUT);
  pinMode(pinoLedVerde, OUTPUT);
  pinMode(pinoBuzzer, OUTPUT);
  digitalWrite(pinoLedVerde, LOW);
  digitalWrite(pinoLedVermelho, LOW);
  digitalWrite(pinoBuzzer, LOW);
  conectarWiFi();
  client.setServer(mqtt_server, mqtt_port);
  client.setCallback(callbackMQTT);
  Serial.println("Calibrando sensor PIR (30 segundos)...");
  delay(30000);
  digitalWrite(pinoLedVerde, HIGH);
  digitalWrite(pinoLedVermelho, HIGH);
  delay(1000);
  digitalWrite(pinoLedVerde, LOW);
  digitalWrite(pinoLedVermelho, LOW);
  Serial.println("Aguardando comando ON...");
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    unsigned long agora = millis();
    if (agora - ultimaTentativaReconexaoWifi >= intervaloReconexaoWifi) {
      ultimaTentativaReconexaoWifi = agora;
      conectarWiFi();
    }
  }
  if (WiFi.status() == WL_CONNECTED && !client.connected()) {
    unsigned long agora = millis();
    if (agora - ultimaTentativaReconexaoMqtt >= intervaloReconexaoMqtt) {
      ultimaTentativaReconexaoMqtt = agora;
      conectarMQTT();
    }
  }
  client.loop();
  if (sistemaAtivo) {
    int leituraPIR = digitalRead(pinoPIR);
    if (leituraPIR != estadoAnteriorPIR) {
      estadoAnteriorPIR = leituraPIR;
      if (leituraPIR == HIGH) {
        digitalWrite(pinoLedVermelho, HIGH);
        digitalWrite(pinoLedVerde, LOW);
        digitalWrite(pinoBuzzer, HIGH);
        publicarTelemetria(true);
      } else {
        digitalWrite(pinoLedVermelho, LOW);
        digitalWrite(pinoLedVerde, HIGH);
        digitalWrite(pinoBuzzer, LOW);
        publicarTelemetria(false);
      }
    }
  }
  delay(100);
```

```
}

void conectarWiFi() {
  WiFi.begin(ssid, password);
  int tentativas = 0;
  while (WiFi.status() != WL_CONNECTED && tentativas < 20) {
    delay(500);
    tentativas++;
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("Wi-Fi conectado!");
  }
}

void conectarMQTT() {
  if (client.connect(mqtt_client_id)) {
    client.subscribe(topico_comando_sistema);
  }
}

void callbackMQTT(char* topico, byte* payload, unsigned int tamanho) {
  String mensagem = "";
  for (unsigned int i = 0; i < tamanho; i++) {
    mensagem += (char)payload[i];
  }
  if (String(topico) == topico_comando_sistema) {
    if (mensagem == "ON") {
      sistemaAtivo = true;
      estadoAnteriorPIR = LOW;
      digitalWrite(pinoLedVerde, HIGH);
      digitalWrite(pinoLedVermelho, LOW);
      digitalWrite(pinoBuzzer, LOW);
    } else if (mensagem == "OFF") {
      sistemaAtivo = false;
      digitalWrite(pinoLedVerde, LOW);
      digitalWrite(pinoLedVermelho, LOW);
      digitalWrite(pinoBuzzer, LOW);
    }
  }
}

void publicarTelemetria(bool detectado) {
  if (!client.connected()) return;
  String mensagem = "{";
  mensagem += "\"deviceId\":\"esp32-alarme-01\",";
  mensagem += "\"sensorPIR\":";
  mensagem += detectado ? "true" : "false";
  mensagem += ",\"timestamp\":";
  mensagem += millis();
  mensagem += "}";
  client.publish(topico_telemetria, mensagem.c_str());
}
```

7. EVIDÊNCIA DA COMUNICAÇÃO BIDIRECIONAL

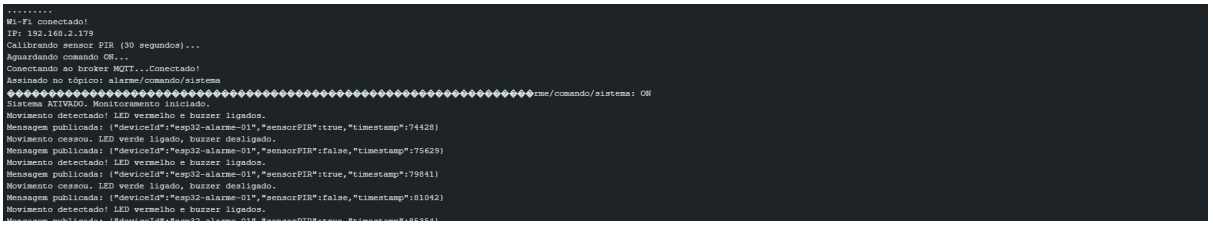
Para validar a comunicação bidirecional, o comando de ativação foi publicado manualmente através do cliente de linha de comando do próprio broker Mosquitto, executado dentro do container Docker:

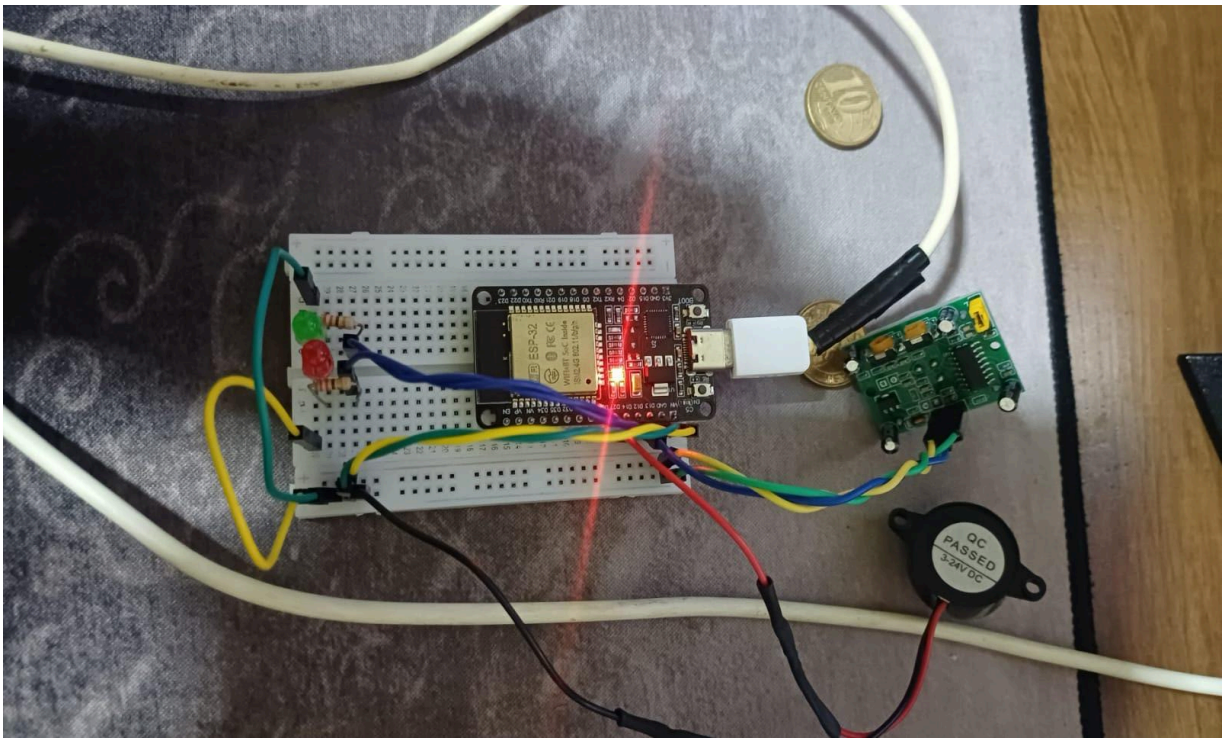
docker exec -it mosquitto-broker-iot mosquitto_pub -t "alarme/comando/sistema" -m "ON"

O Serial Monitor do ESP32 confirmou o recebimento do comando e o funcionamento completo do fluxo bidirecional: recebimento do comando ON, ativação do sistema, detecção de movimento pelo sensor PIR, acionamento dos atuadores (LED vermelho e buzzer) e publicação da telemetria correspondente. Trecho do log observado:

```
Wi-Fi conectado!
IP: 192.168.2.179
Calibrando sensor PIR (30 segundos)...
Aguardando comando ON...
Conectando ao broker MQTT...Conectado!
Assinado no tópico: alarme/comando/sistema
Sistema ATIVADO. Monitoramento iniciado.
Movimento detectado! LED vermelho e buzzer ligados.
Mensagem publicada:
{"deviceId":"esp32-alarme-01","sensorPIR":true,"timestamp":74428}
Movimento cessou. LED verde ligado, buzzer desligado.
Mensagem publicada:
{"deviceId":"esp32-alarme-01","sensorPIR":false,"timestamp":75629}
Movimento detectado! LED vermelho e buzzer ligados.
Mensagem publicada:
{"deviceId":"esp32-alarme-01","sensorPIR":true,"timestamp":79841}
```

O ciclo completo (ON → ativação → detecção → telemetria → OFF → desativação) foi testado e confirmado com sucesso, demonstrando que o ESP32 é capaz de, simultaneamente, publicar dados de sensores e receber comandos externos pelo mesmo canal MQTT.





Protótipo já montado com sensor PIR, o buzzer e LEDs de indicação.